TIP It's possible to write your own authorization filters by implementing `IAuthorizationFilter` or `IAsyncAuthorizationFilter`, but I strongly advise against it. The ASP.NET Core authorization framework is highly configurable and should meet all your needs.

At the heart of MVC authorization is an authorization filter, `AuthorizeFilter`, which you can add to the filter pipeline by decorating your actions or controllers with the `[Authorize]` attribute. In its simplest form, adding the `[Authorize]` attribute to an action, as in the following listing, means that the request must be made by an authenticated user to be allowed to continue. If you're not logged in, it will short-circuit the pipeline, returning a `401 Unauthorized` response to the browser.

Listing 22.3 Adding `[Authorize]` to an action method

```
public class RecipeApiController : ControllerBase
{
    public IActionResult Get(int id)                          ❶
    {
        // method body
    }

    [Authorize]                                               ❷
    public IActionResult Edit(                                ❸
        int id, [FromBody] UpdateRecipeCommand command)       ❸
    {
        // method body
    }
}
```

❶ The Get method has no [Authorize] attribute, so it can be executed by anyone.

❷ Adds the AuthorizeFilter to the filter pipeline using [Authorize]

❸ The Edit method can be executed only if you're logged in.